本地化、事件驱动型MT5高频跟单系统：开发者终极技术规程

1. 系统规程概述

本文档为一套专为本地化部署设计、事件驱动、以Python为核心的MT5高频跟单系统的最终技术实现规程。本文档面向开发人员，提供从环境配置到模块化代码实现的完整、精确的技术细节，旨在构建一个功能完备、性能卓越且高度可扩展的交易工具。

2. 技术栈与环境配置

2.1. 核心技术栈

- **MQL5**: 用于主账户端事件捕获EA。
- **Python**: 3.9+，作为核心控制器和GUI的开发语言。
- **MetaTrader5 Python**包: MetaTrader5，用于Python与MT5终端的交互 [2]。
- 进程间通信 **(IPC)**: pyzmq，使用ZeroMQ库实现MQL5与Python之间的高性能、低延迟消息传递 [2]。
- 图形用户界面 **(GUI)**: PyQt5，用于构建功能丰富的桌面控制应用 [2]。
- 配置文件: JSON，用于存储和管理所有账户及策略配置 [2]。

2.2. 开发环境配置规程

1. 安装**MT5**终端:
    - 安装一个标准的MT5终端，用于主账户。

    - 为每个副账户（例如，副账户1和副账户2），完整复制主账户的MT5安装目录，创建独立的文件夹（如 C:\MT5_Slave_1, C:\MT5_Slave_2）。
    - 为每个MT5实例（主账户和所有副账户）创建桌面快捷方式。右键点击快捷方式 -> 属性 -> 在“目标”字段的末尾添加空格和 /portable 参数。此步骤至关重要，它能确保每个MT5实例拥有独立的配置和数据，是实现多账户本地连接的基础 [2]。
2. 安装**Python**及库:
    - 安装Python 3.9或更高版本。
    - 使用pip安装必要的库: pip install MetaTrader5 pyzmq PyQt5。
3. 配置**ZeroMQ**库:
    - 从ZeroMQ官方或可信来源下载与MT5终端位数（64位）匹配的libzmq.dll文件。
    - 将libzmq.dll文件精确放置到主账户**MT5**安装目录下的MQL5\Libraries文件夹内。
4. 配置**MT5**终端选项:
    - 启动主账户**MT5**终端。
    - 进入“工具” -> “选项” -> “EA交易”选项卡。
    - 必须勾选以下两个选项：“允许Algo Trading”和“允许从URL列表导入DLL” [2]。这是允许EA运行和调用外部 libzmq.dll的前提。

3. 整体系统架构

3.1. 模块化架构

系统严格遵循三层模块化架构，各模块在同一台机器上通过本地IPC高效协同 [2]。

- **L1 - MQL5**事件发布器 **(Event Publisher)**: 部署在主账户MT5上的轻量级EA，仅负责通过OnTradeTransaction捕获交易事件并立即通过ZMQ发布。
- **L2 - Python**核心控制器 **(Core Controller)**: 独立的Python后台逻辑，订阅ZMQ事件，管理所有副账户连接，并执行全部跟单策略。
- **L3 - Python**图形用户界面 **(GUI)**: 系统的控制与监控中心，与核心控制器解耦，通过独立的ZMQ信道或Qt信号/槽机制进行通信。

3.2. 系统数据流图



4. 核心组件实现细则

4.1. MQL5 事件发布器 (TradePublisher.mq5)

- 事件钩子 **(Event Hook)**: 必须使用OnTradeTransaction作为唯一的事件捕获入口。此函数由终端在交易事务发生时立即回调，确保零延迟捕获 [2]。需要处理的关键trans.type包括：
  - TRADE_TRANSACTION_DEAL_ADD: 核心事件，表示新成交（市价单执行或挂单触发）。
  - TRADE_TRANSACTION_ORDER_ADD: 新挂单创建。
  - TRADE_TRANSACTION_ORDER_UPDATE: 挂单修改。
  - TRADE_TRANSACTION_ORDER_DELETE: 挂单取消。
- 通信协议: 使用ZMQ的PUB套接字绑定到本地地址（如tcp://*:5556）。
- 数据完整性 - 同步握手: 为防止Python控制器启动慢于EA导致消息丢失，必须实现REQ/REP同步握手 [2]。
  1. EA在OnInit中，除了创建PUB套接字，还需创建并绑定一个REP套接字（如tcp://*:5555）。
  2. EA在此REP套接字上阻塞等待，直到收到来自Python控制器的"SYNC"消息。
  3. 收到"SYNC"后，EA回复"ACK"，然后才开始在OnTradeTransaction中通过PUB套接字发布交易事件。
- 数据契约 **(JSON Schema)**: 所有发布的交易事件必须遵循以下严格的JSON结构，通过多部分消息发送（第一部分为主题b"TRADE_EVENT"，第二部分为JSON负载） [2]。

字段名	数据类型	MQL5来源	描述
timestamp_msc	ulong	trans.time_msc	事件发生的毫秒级时间戳，用于延迟计算和排序。
event_type	string	EnumToString(trans.type)	交易事务类型，如"TRADE_TRANSACTION_DEAL_ADD" [2]。
position_id	ulong	trans.position	关联的持仓ID (ticket)。
order_id	ulong	trans.order	关联的订单ID (ticket)。
deal_id	ulong	trans.deal	关联的成交ID (ticket)。

symbol	string	trans.symbol	金融产品代码。
volume	double	trans.volume	交易量（手数）。
price	double	trans.price	交易价格。
sl	double	trans.sl	止损价格。
tp	double	trans.tp	止盈价格。
order_type	int	request.type	原始订单类型（如 ORDER_TYPE_BUY, ORDER_TYPE_SELL_LIMIT）。
magic_number	ulong	request.magic	原始交易的魔术号。
comment	string	request.comment	原始交易的注释。

4.2. Python 核心控制器

- 并发模型: 采用单后台线程模型。使用PyQt5的QThread，将所有后台逻辑（ZMQ监听、账户管理、交易执行）封装在一个CoreWorker(QObject)对象中，并将其moveToThread到一个QThread实例。此模型对于I/O密集型的跟单任务足够高效，且能与GUI通过信号/槽机制安全、简单地集成，避免了多进程带来的复杂状态同步问题 [2]。
- 模块化类结构:
  - **CoreWorker(QObject)**: 后台任务的执行者，包含其他核心类的实例，并在其run方法中启动ZMQ监听循环。
  - **AccountManager**: 负责从config.json加载账户配置，并使用mt5.initialize(path=...)和mt5.login(...)为每个副账户建立并维护一个MT5连接实例。提供get_connection(account_id)等接口。
  - **ZmqListener**: 封装ZMQ通信逻辑。在CoreWorker启动时，首先执行REQ/REP同步握手，然后创建SUB套接字订阅b"TRADE_EVENT"主题，并进入socket.recv_multipart()的阻塞循环。
  - **TradeEngine**: 核心跟单逻辑引擎。
    - 状态管理: 必须维护一个精确的主副账户订单/持仓映射表：trade_map = {master_ticket: {slave_account_id: slave_ticket}}。此表是实现订单修改和关闭同步的关键 [2]。
    - 事件处理: 提供process_event(event_data)方法，根据事件类型调用不同的私

有处理函数（如_handle_new_deal, _handle_modify_order）。

4.3. Python 图形用户界面 (GUI)

- 框架与设计: 使用PyQt5。UI界面在**Qt Designer**中进行可视化设计，并保存为.ui文件。在Python代码中使用uic.loadUi()动态加载，实现UI与逻辑的完全分离 [2]。
- 线程安全: 严格使用信号/槽机制进行跨线程通信。CoreWorker中定义各种信号（如log_signal = pyqtSignal(str), status_update_signal = pyqtSignal(dict)），在GUI主窗口中将这些信号连接到对应的UI更新槽函数。严禁在CoreWorker中直接操作任何GUI控件 [2]。
- **UI**功能实现:
  - 账户与策略管理 **(QTableView)**:
    - 实现一个自定义的QAbstractTableModel来与config.json数据双向绑定。
    - 表格中的QComboBox（跟单模式、手数模式）和QCheckBox（启用、同步SL/TP）的交互需要连接到模型的setData方法，以实时更新配置。
    - “添加/删除账户”按钮直接操作数据模型，并触发config.json的重写。
  - 运行控制与监控:
    - “启动/暂停/停止”按钮通过发射信号或调用CoreWorker的槽函数来控制后台服务的状态。
    - 实时持仓和账户状态的QTableView和QLabel由CoreWorker发出的信号驱动更新。
  - 日志: QPlainTextEdit控件连接到CoreWorker的log_signal，实现日志的实时滚动追加。

5. 核心跟单逻辑实现

5.1. 交易事件分发

ZmqListener接收到消息后，TradeEngine的process_event方法作为总入口，根据event_type将事件分发至具体的处理函数。

5.2. 市价单与挂单处理

- 正向跟单:
  - 市价单: BUY -> BUY, SELL -> SELL。
  - 挂单: BUY_LIMIT -> BUY_LIMIT, SELL_STOP -> SELL_STOP。
- 反向跟单 **(**精确转换**)** [2]:
  - 市价单: BUY -> SELL, SELL -> BUY。
  - 挂单:
    - BUY_LIMIT -> SELL_STOP
    - SELL_LIMIT -> BUY_STOP
    - BUY_STOP -> SELL_LIMIT
    - SELL_STOP -> BUY_LIMIT

5.3. 止损**/**止盈 **(SL/TP)** 处理

- 正向跟单: 直接复制主账户的SL/TP价格。
- 反向跟单: 严格交换SL和TP。slave_sl = master_tp, slave_tp = master_sl [2]。

5.4. 订单修改与平仓

- 依赖映射表: 所有修改和平仓操作都必须首先通过trade_map查找主账户ticket对应的副账户ticket。
- 修改挂单: 主账户TRADE_TRANSACTION_ORDER_UPDATE事件触发后，为每个副账户构建一个request字典，设置action为mt5.TRADE_ACTION_MODIFY，并传入副账户的ticket来修改。
- 取消挂单: 主账户TRADE_TRANSACTION_ORDER_DELETE事件触发后，构建request字典，设置action为mt5.TRADE_ACTION_REMOVE来删除副账户的挂单。
- 平仓: 主账户平仓事件（一个反向的DEAL_ADD）触发后，为每个副账户构建一个与其持仓方向相反的市价单请求（TRADE_ACTION_DEAL）来平掉对应持仓。

5.5. 手数计算策略

在TradeEngine中实现，根据GUI为每个账户配置的lot_mode进行计算。

- 固定手数: 直接使用配置值。
- 手数倍数: slave_lot = master_lot * multiplier。
- 风险百分比: Lots = (AccountBalance * RiskPercent) / (StopLossPips * PipValue)。此计算需要额外获取账户信息和品种信息。

6. 风险控制与过滤机制

这些机制是系统健壮性的关键，必须在TradeEngine处理事件时强制执行 [2]。

- 延迟过滤 (秒偏差): 在处理DEAL_ADD事件时，比较当前时间与消息中的timestamp_msc。如果差值超过GUI中设定的“允许秒偏差”，则拒绝跟单。
- 滑点过滤 (点偏差): 在跟单市价单前，获取副账户的当前市价。如果当前市价与主账户成交价相比，已向不利方向移动超过GUI设定的“允许点偏差”，则拒绝跟单。
- 挂单同步选项: GUI提供“跟挂单”复选框。如果未勾选，TradeEngine将忽略所有ORDER_ADD, ORDER_UPDATE, ORDER_DELETE事件，仅在主账户挂单触发（产生DEAL_ADD事件）后，才在副账户以市价跟进。
- SL/TP同步选项: GUI提供“跟止损止盈价”复选框。如果未勾选，TradeEngine在构建所有交易请求时，将sl和tp字段设置为0。

7. 部署与文件结构

7.1. 推荐文件结构

```
/mt5_copy_trader/
|-- main.py               # 程序主入口，初始化GUI和CoreWorker
|-- config.json           # 存储所有账户和策略配置
|
|-- /mql5_ea/
| |-- TradePublisher.mq5    # MQL5事件发布器源码
|
|-- /core/
| |-- __init__.py
| |-- worker.py           # 定义CoreWorker类
| |-- account_manager.py    # 定义AccountManager类
| |-- trade_engine.py       # 定义TradeEngine类
| |-- zmq_listener.py       # 定义ZmqListener类
|
|-- /gui/
| |-- __init__.py
| |-- main_window.py        # GUI主窗口类
| |-- account_table_model.py # 账户表格的自定义模型
| |-- main_window.ui        # Qt Designer设计的UI文件
|
|-- /logs/
| |-- trade_copier.log      # 日志文件
```

7.2. config.json 最终结构

此文件由GUI的账户管理模块完全控制。

JSON

```
{
  "accounts":
}
```

8. 附录：MQL5与Python核心代码框架

8.1. TradePublisher.mq5 核心框架

C++

```
// MQL5 MasterPublisher.mq5
#include <Zmq/Zmq.mqh> // 社区提供的ZMQ绑定库
//... (包含JSON序列化库)...

//--- 输入参数
input string SyncEndpoint  = "tcp://*:5555";
input string PubEndpoint   = "tcp://*:5556";

//--- 全局变量
void* zmq_context;
void* zmq_publisher;
void* zmq_sync_service;
bool  is_synced = false;

int OnInit() {
   // 1. 初始化ZMQ上下文和套接字
   zmq_context = ZmqContextCreate();
   zmq_publisher = ZmqSocketCreate(zmq_context, ZMQ_PUB);
   ZmqBind(zmq_publisher, PubEndpoint);
   zmq_sync_service = ZmqSocketCreate(zmq_context, ZMQ_REP);
   ZmqBind(zmq_sync_service, SyncEndpoint);

   // 2. 在一个非阻塞的定时器中处理同步，避免OnInit卡死
   EventSetTimer(1); // 每秒检查一次
   return(INIT_SUCCEEDED);
```

```
}

void OnTimer() {
   if (is_synced) {
      // 已同步，可以发送心跳或停止定时器
      EventKillTimer();
      return;
   }

   uchar sync_request[1];
   int n = ZmqRecv(zmq_sync_service, sync_request, 4, ZMQ_NOBLOCK);
   if (n > 0 && CharArrayToString(sync_request) == "SYNC") {
      Print("Sync request received. Sending ACK.");
      uchar sync_reply;
      StringToCharArray("ACK", sync_reply);
      ZmqSend(zmq_sync_service, sync_reply, StringLen("ACK"));
      is_synced = true;
      Print("Synchronization complete. Ready to publish trade events.");
   }
}

void OnDeinit(const int reason) {
   if(zmq_publisher) ZmqClose(zmq_publisher);
   if(zmq_sync_service) ZmqClose(zmq_sync_service);
   if(zmq_context) ZmqTerm(zmq_context);
}

void OnTradeTransaction(const MqlTradeTransaction &trans, const MqlTradeRequest &request, const
MqlTradeResult &result) {
   if (!is_synced) return; // 未同步则不发送任何消息

   // 1. 过滤不感兴趣的事件
   //...

   // 2. 构建JSON字符串 (使用JSON库)
   string json_payload = "{\"timestamp_msc\":" + (string)trans.time_msc + ",... }";

   // 3. 发送多部分消息
   uchar topic, payload;
```

```
   StringToCharArray("TRADE_EVENT", topic);
   StringToCharArray(json_payload, payload);
   ZmqSend(zmq_publisher, topic, ArraySize(topic) - 1, ZMQ_SNDMORE);
   ZmqSend(zmq_publisher, payload, ArraySize(payload) - 1);
}
```

8.2. main.py Python应用入口框架

Python

```
# main.py
import sys
from PyQt5.QtWidgets import QApplication
from gui.main_window import MainWindow
from core.worker import CoreWorker

def main():
    # 1. 初始化Qt应用
    app = QApplication(sys.argv)

    # 2. 创建后台工作器和GUI窗口实例
    # CoreWorker负责所有后台逻辑，它将运行在独立的QThread中
    worker = CoreWorker(config_path='config.json')
    main_window = MainWindow(worker) # GUI可以持有worker的引用以调用其槽函数

    # 3. 建立信号和槽的连接
    # 将worker的信号连接到main_window的槽，用于更新UI
    worker.log_signal.connect(main_window.update_log)
    worker.status_signal.connect(main_window.update_status)
    # 将main_window的信号连接到worker的槽，用于从UI控制后台
    main_window.start_signal.connect(worker.start)
    main_window.stop_signal.connect(worker.stop)
    main_window.pause_signal.connect(worker.pause)
```

```
    # 4. 启动后台工作器线程
    worker.start_thread()

    # 5. 显示GUI并启动事件循环
    main_window.show()
    sys.exit(app.exec_())

if __name__ == "__main__":
    main()
```

Works cited

1. Automated Trading using MT5 and Python - Quantra by QuantInsti, accessed on June 15, 2025, https://quantra.quantinsti.com/glossary/Automated-Trading-using-MT5-and-Python
2. 新速编程 MT4-MT4跟单EA使用说明.pdf